Digital Image Processing Lab (NCSH 302) **Session: Monsoon 2026-27**
B. Tech. 5th Sem (CSE-Hons) **Date: 11.08.2026**

Assignment-1 (Pixel Geometry)

1. For an image $Im(x, y)$, the $4-neighbors$ of a pixel $p \in Im$ are denoted by $N_4(p)$ located at locations $(x-1, y), (x+1, y), (x, y-1), and\ (x, y+1)$, whereas $4-diagonal$ neighbors of p, denoted by $N_D(p)$ are located at $(x-1, y-1), (x+1, y+1), (x+1, y-1), and\ (x-1, y+1)$ locations. Further, $8-neighbors$ of pixel p is the union of $4-neighbors\ of\ p\ and\ 4-diagonal\ neighbors\ of\ p$ which can be denoted as $N_8(p) = N_4(p) \cup N_D(p)$. For a set of intensity $values\ V$, $4-adjacency, 8-adjacency, and\ m-adjacency$ can be defined as following:

   - $4-$ ***adjacency***: Two $pixels\ p\ and\ q$ with values from V are $4-adjacent$ if q is in the set $N_4(p)$.
   - $\mathbf{8-}$ ***adjacency***: Two $pixels\ p\ and\ q$ with values from V are $8-adjacent$ if q is in the set $N_8(p)$.
   - $m-$ ***adjacency***: Two $pixels\ p\ and\ q$ with values from V are $m-adjacent$ if q is in the set $N_4(p)$, or q is in the set $N_D(p)$ and the set $N_4(p) \cap N_4(q)$ has no pixels whose values are from V.

   Consider, you have been given an image $Im\ (x, y)$ with gray-scale levels $L = [0, 1, 2, 3, 4, 5, 6, 7]$ such that position of top-leftmost and bottom-rightmost pixel is $(1, 1)$ and $(8, 8)$, respectively. Write a function $Pixel_adj\ (Im, adj, p)$ to find the adjacent pixels as specified by adj for a given pixel $p \in Im$ where adj can take the value as $4-adjacency, 8-adjacency, and\ m-adjacency$ such that $V = \{0,1\}$. If the chosen pixel lies on the boundary, then to handle this case replicate the boundary values by adding row or column above or below in the image as per the requirement.

Sample Input:

Input Image

3	5	3	5	2	4	3	2
4	6	6	7	3	3	7	6
6	3	0	3	1	3	2	3
5	4	2	1	2	3	6	7
2	7	3	0	0	1	2	5
7	4	1	0	2	3	3	4
4	2	7	3	5	7	2	5
2	6	5	4	2	7	5	6

Position of given pixel p = (4, 4)

Sample Output:

4 – adjacency:
Value = 0 position = (5, 4)

8 – adjacency:
Value = 0 position = (5, 4)
Value = 0 position = (5, 5)
Value = 0 position = (3, 3)
Value = 1 position = (3, 5)

m – adjacency:
Value = 0 position = (3, 3)
Value = 1 position = (3, 5)
Value = 0 position = (5, 4)

2. A path from $pixel\ p$ with coordinates (x_0, y_0) to pixel q with coordinates (x_n, y_n) is a sequence of pixels with coordinates $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$ where (x_i, y_i) and (x_{i-1}, y_{i-1}) are adjacent for $1 \leq i \leq n$, where n represents the length of the path. We can define $4-, 8-, and\ m-paths$ based on the type of $adjacency$ used. Suppose, you have been provided a gray-scale image $Im(x, y)$ with gray-scale levels $L = [0,1,2,3,4,5,6,7]$ of dimension $W \times H$ where W and H represent the number of rows and columns of image pixels, respectively. Consider, for given image, $top-leftmost$ and $bottom-rightmost$ pixels are located at locations $(1,1)$ and $(7,7)$, respectively. Write a program that finds different paths between two user-given points $P(x, y)$ and $Q(x', y')$ where set of intensity values V is $\{0, 1\}$. Your program should be able to display all possible $4-paths, 8-paths, and\ m-paths$ between the user-given points.

Sample Input:

Input Image

3	7	6	5	7	3	6
3	2	3	2	4	5	7
5	3	5	1	0	2	5
7	4	6	1	6	5	3
6	2	8	3	0	7	2
4	6	7	2	3	4	7
2	5	4	6	5	4	6

Enter the location of first point p = (3, 5)
Enter the location of second point q = (5, 4)

Sample Output:

No 4 - path exists.

Possible 8 – paths are:
(3, 5), (3, 4), (4, 4), (5, 5)
Path Length: 3
(3, 5), (4, 4), (5, 5)
Path Length: 2

Possible m – paths are:
(3, 5), (3, 4), (4, 4), (5, 5)
Path Length: 3

3. Let S represents a subset of pixels in an image Im. Two $pixels\ p\ and\ q$ are said to be connected in S if there exists a path between them consisting entirely of pixels in S. For every $pixel\ p\ in\ S$, the set of pixels in S that are connected to p is called a connected component of S. Write a program to compute the connected components present in user given $gray-scale$ image Im by applying following steps.

   i. At first, write a function Im_{binary} = gray_to_binary (Im, T) which can convert the user-given $gray-scale$ image Im into binary image Im_{binary} by performing the following operation over every pixel p of image Im.

$$\begin{cases} p = 1, & if\ p \geq T \\ p = 0, & otherwise \end{cases}$$

   Where T is the user-given threshold value.

Sample Input: **Sample Output:**





$T = 0.5$ (A threshold value of 0.5 corresponds to an intensity value halfway between the minimum and maximum value of the image pixel intensity.)

ii. In next step, write a function $Connected_components\ (Im_{binary}, con)$ which computes number of connected components present in the binary image Im_{binary} where con represents the $connectivity$ specified by user.

Sample Input:

$Con = 4$
$Con = 8$

Sample Output:

For $4 - connectivity$, number of connected components are: 2739
For $8 - connectivity$, number of connected components are: 1740

iii. As last operation, find out the largest connected components of the binary image Im_{binary} corresponding to both $4 - connectivity$ and $8 - connectivity$ criteria. Represent the outcome obtained by setting the pixel values to 0 corresponding to the largest connected component within image Im_{binary}.

Sample Output:



For $4 - connectivity$



For $8 - connectivity$

4. Let $Im_{binary}(x, y)$ be a binary image and let S denote the set of $foreground\ (1 - valued)$ pixels in it. Suppose S consists of $K\ disjoint\ connected\ components\ (regions)$ $R_1, R_2, \dots, R_K$, obtained using a specified connectivity $con(4 - or\ 8 - connectivity)$. Two regions $R_i\ and\ R_j$ are said to be adjacent if their union $R_i \cup R_j$ forms a connected set; otherwise, they are disjoint. The boundary of a region R is the set of pixels in R that have

one or more neighbors not in R. Given a binary image $Im_{binary}(x, y)$ containing its K regions, write the following functions:

i. adj_{flag} = Region_adjacency $\left(Im_{binary},\ R_i,\ R_j\right)$ – returns 1 if regions R_i and R_j are adjacent, and 0 if they are disjoint.

ii. $Im_{boundary}$ = Boundary_extract $\left(Im_{binary}, con\right)$ – extracts the boundary pixels of every region present in Im_{binary} using the connectivity con and reports the boundary pixel count separately for each region.

iii. Im_{fg}, Im_{bg} = Foreground_background $\left(Im_{binary}\right)$ – separates and returns the foreground image (union of all regions) and the background image (its complement) and reports the total pixel count of each.

Sample Input:

Binary Image (8 × 8):

0	0	0	0	0	0	0	0
0	1	1	0	0	0	0	0
0	1	1	0	0	1	1	0
0	0	0	0	1	1	0	0
0	0	0	0	0	0	0	0
0	0	1	1	0	0	0	0
0	0	1	1	0	0	0	0
0	0	0	0	0	0	0	0

Sample Input:

Number of connected regions = 3
Region 1 = 4 pixels
Region 2 = 4 pixels
Region 3 = 4 pixels

$Region_adjacency\ (R_1, R_2)$ = 0

Boundary pixel count:
Region 1 = 4
Region 2 = 4
Region 3 = 4

Foreground pixel count = 12
Background pixel count = 52